네트워크 관리 모듈 분석

개요

Oasis CLI의 네트워크 관리 모듈은 다양한 Oasis 네트워크와의 연결 및 상호작용을 담당하는 핵심 구성 요소입니다. 이 모듈은 네트워크 설정 관리, RPC 연결 설정, 체인 컨텍스트 관리, 그리고 네트워크 상태 조회 기능을 제공합니다.

아키텍처 구조

명령어 계층 구조

```
network (cmd/network/)
├── add - 새 네트워크 추가
├── add-local - 로컬 네트워크 추가
├── list - 설정된 네트워크 목록 조회
├── remove - 네트워크 제거
├── set-chain-context - 체인 컨텍스트 설정
├── set-default - 기본 네트워크 설정
├── set-rpc - RPC 엔드포인트 설정
├── show - 네트워크 속성 조회
├── status - 네트워크 상태 조회
└── governance/ - 거버넌스 관련 명령어
```

핵심 구성 요소

1. **네트워크 설정 관리자** (`config/config.go`)
2. **연결 관리자** (SDK의 `connection` 패키지)
3. **명령어 처리기** (`cmd/network/` 디렉토리)
4. **설정 마이그레이션** (`config/migrate.go`)

네트워크 설정 관리

설정 구조

네트워크 설정은 다음과 같은 계층 구조를 가집니다:

```
type Config struct {
Networks config.Networks `mapstructure:"networks"`
// ... 기타 설정
}


// SDK에서 정의된 네트워크 구조
type Network struct {
ChainContext  string
RPC  string
Description  string
Denomination Denomination
ParaTimes ParaTimes
```

```
}
```

기본 네트워크 설정

CLI는 다음과 같은 사전 정의된 네트워크를 제공합니다:

- **Mainnet**: 프로덕션 네트워크
- **Testnet**: 테스트 네트워크

이러한 기본 설정은 `config.DefaultNetworks` 에서 관리되며, 자동 마이그레이션을 통해 업데이트됩니다.

설정 파일 위치

네트워크 설정은 다음 위치에 TOML 형식으로 저장됩니다:

- 경로: `$XDG_CONFIG_HOME/oasis/cli.toml`
- 형식: TOML
- 인코딩: UTF-8

네트워크 추가 및 관리

일반 네트워크 추가 (`add` 명령어)

```go
// cmd/network/add.go의 핵심 로직
func addNetworkCommand(name, rpc string, chainContext string) {
// 1. 네트워크 설정 검증
cobra.CheckErr(config.ValidateIdentifier(name))
// 2. 네트워크 객체 생성
```

```
net := config.Network{RPC: rpc}

// 3. 체인 컨텍스트 자동 조회 (제공되지 않은 경우)

if chainContext == "" {

conn, err := connection.ConnectNoVerify(ctx, &network)

chainCtx, err := conn.Consensus().Core().GetChainContext(ctx)

net.ChainContext = chainCtx

}

// 4. 사용자 입력을 통한 추가 정보 수집

networkDetailsFromSurvey(&net)

// 5. 설정에 추가 및 저장

cfg.Networks.Add(name, &net)

cfg.Save()

}
```

로컬 네트워크 추가 (`add-local` 명령어)

로컬 네트워크 추가는 다음과 같은 특별한 처리를 포함합니다:

1. **Unix 소켓 지원**: `unix:` 프로토콜 자동 감지
2. **절대 경로 변환**: 상대 경로를 절대 경로로 변환
3. **기본 설정 복제**: 알려진 체인 컨텍스트의 경우 기본 설정 자동 적용

```
// 로컬 엔드포인트 절대 경로 추출

func extractAbsPath(rpc string) (string, error) {

extracted := strings.Split(rpc, ":")
```

```
scheme, path := extracted[0], extracted[1]

// 홈 디렉토리 확장

if strings.HasPrefix(path, "~/") {

home, _ := os.UserHomeDir()

path = filepath.Join(home, path[2:])

}

// 절대 경로 변환

absPath, err := filepath.Abs(path)

return strings.Join([]string{scheme, absPath}, ":"), nil

}
```

RPC 연결 및 체인 컨텍스트 관리

연결 설정 과정

1. **연결 검증**: `connection.Connect()` 또는 `connection.ConnectNoVerify()`
2. **체인 컨텍스트 조회**: `conn.Consensus().Core().GetChainContext()`
3. **네트워크 검증**: `net.Validate()`

체인 컨텍스트 관리

체인 컨텍스트는 네트워크의 고유 식별자로 사용됩니다:

```
// 체인 컨텍스트 설정 또는 자동 조회

func setChainContext(name string, chainContext string) {

net := cfg.Networks.All[name]
```

```
if chainContext != "" {
net.ChainContext = chainContext
} else {
// 네트워크에 연결하여 자동 조회
network := config.Network{RPC: net.RPC}
conn, err := connection.ConnectNoVerify(ctx, &network)
chainCtx, err := conn.Consensus().Core().GetChainContext(ctx)
net.ChainContext = chainCtx
}
}
```

RPC 엔드포인트 관리

RPC 엔드포인트는 다음과 같은 형식을 지원합니다:

- **gRPC over TLS**: `grpc.oasis.io:443`
- **Unix 소켓**: `unix:/path/to/socket`
- **로컬 파일**: 자동으로 `unix:` 프로토콜 추가

네트워크 상태 조회 및 통신

상태 조회 (`status` 명령어)

네트워크 상태 조회는 다음 정보를 제공합니다:

1. **합의 계층 정보**:
   - 노드 상태 및 버전

- 최신 블록 높이 및 해시
- 에포크 정보
- 검증자 상태

2. **ParaTime 정보**:

- 런타임 상태
- 위원회 정보
- 스토리지 상태
- 활성 버전

```
// 상태 조회 핵심 로직
func getNetworkStatus(npa *common.NPASelection) {
conn, err := connection.Connect(ctx, npa.Network)
nodeStatus, err := conn.Control().GetStatus(ctx)
// 합의 계층 정보 출력
consensus := nodeStatus.Consensus
fmt.Printf("Chain context: %s", consensus.ChainContext)
fmt.Printf("Latest height: %d", consensus.LatestHeight)
// ParaTime 정보 출력
for key, runtime := range nodeStatus.Runtimes {
paratimeName := getParatimeName(cfg, key.String())
fmt.Printf("%s (%s):", paratimeName, key.String())
fmt.Printf(" Latest round: %d", runtime.LatestRound)
}
```

```
}
```

네트워크 속성 조회 (show 명령어)

show 명령어는 다양한 네트워크 속성을 조회할 수 있습니다:

- **엔티티 정보**: 검증자 엔티티 및 노드 정보
- **노드 정보**: 노드 역할, 소프트웨어 버전, 런타임 정보
- **런타임 정보**: ParaTime 설정 및 상태
- **검증자 정보**: 활성 검증자 목록
- **네이티브 토큰**: 토큰 공급량, 스테이킹 임계값
- **가스 비용**: 트랜잭션 유형별 가스 비용
- **위원회**: ParaTime 위원회 구성
- **매개변수**: 네트워크 합의 매개변수

설정 마이그레이션

자동 마이그레이션 시스템

CLI는 네트워크 설정의 자동 마이그레이션을 지원합니다:

```
func (cfg *Config) migrateNetworks() bool {
var changes bool
for name, net := range cfg.Networks.All {
defaultCfg, knownDefault := Default.Networks.All[name]
oldNetwork, knownOld := OldNetworks[name]
// 구 체인 컨텍스트를 새 것으로 교체
```

```
for _, oldChainContext := range oldNetwork.ChainContexts {
if net.ChainContext == oldChainContext {
net.ChainContext = defaultCfg.ChainContext
changes = true
}
}
// 구 RPC 엔드포인트를 새 것으로 교체
for _, oldRPC := range oldNetwork.RPCs {
if net.RPC == oldRPC {
net.RPC = defaultCfg.RPC
changes = true
}
}
}
return changes
}
```

마이그레이션 대상

- **체인 컨텍스트**: 네트워크 업그레이드 시 자동 업데이트
- **RPC 엔드포인트**: 더 이상 사용되지 않는 엔드포인트 교체
- **ParaTime 설정**: 새로운 ParaTime 자동 추가
- **합의 denomination**: ParaTime별 denomination 업데이트

에러 처리 및 검증

네트워크 설정 검증

```
func (cfg *Config) Validate() error {
if err := cfg.Networks.Validate(); err != nil {
return fmt.Errorf("failed to validate network configuration: %w", err)
}
return nil
}
```

연결 에러 처리

1. **연결 실패**: 네트워크 연결 불가 시 명확한 에러 메시지
2. **체인 컨텍스트 불일치**: 잘못된 네트워크 연결 시 경고
3. **RPC 엔드포인트 오류**: 잘못된 엔드포인트 형식 검증

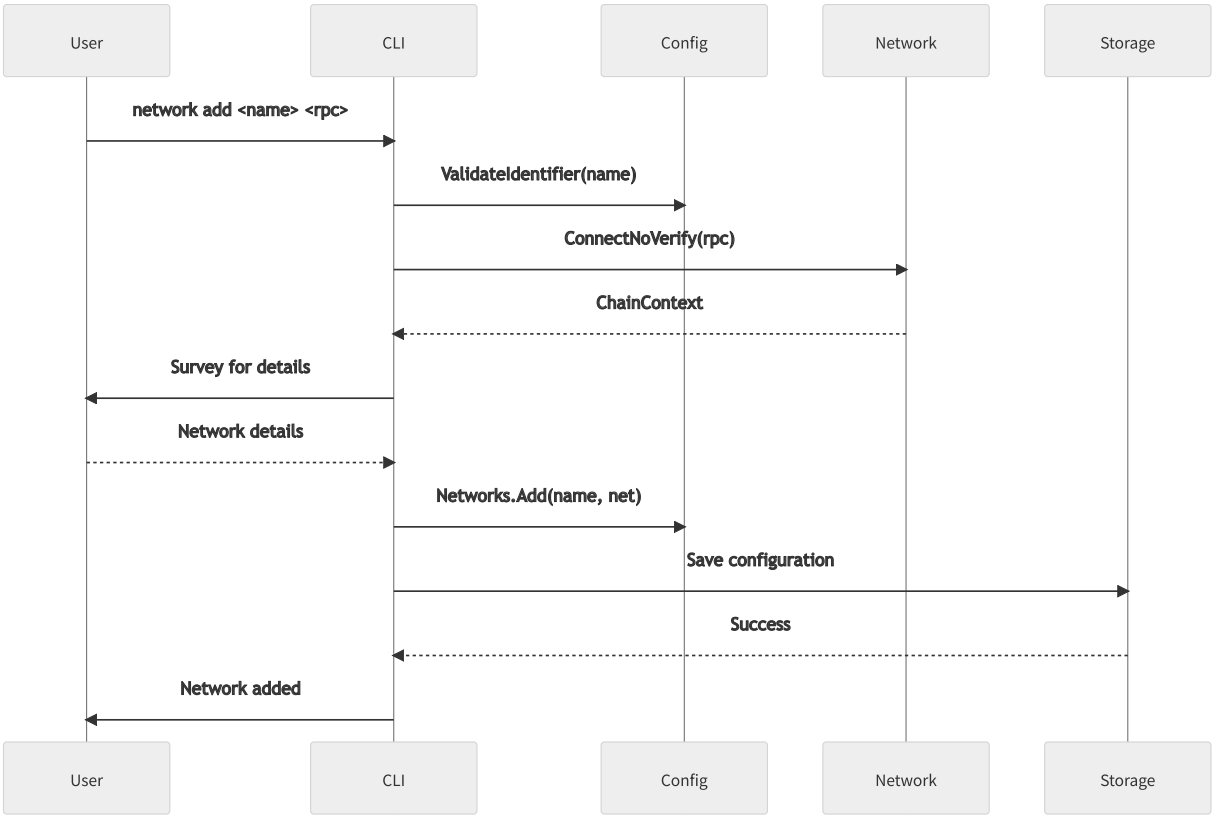
사용자 친화적 에러 메시지

```
// 네트워크 존재 여부 확인
net := cfg.Networks.All[name]
if net == nil {
cobra.CheckErr(fmt.Errorf("network '%s' does not exist", name))
}
```

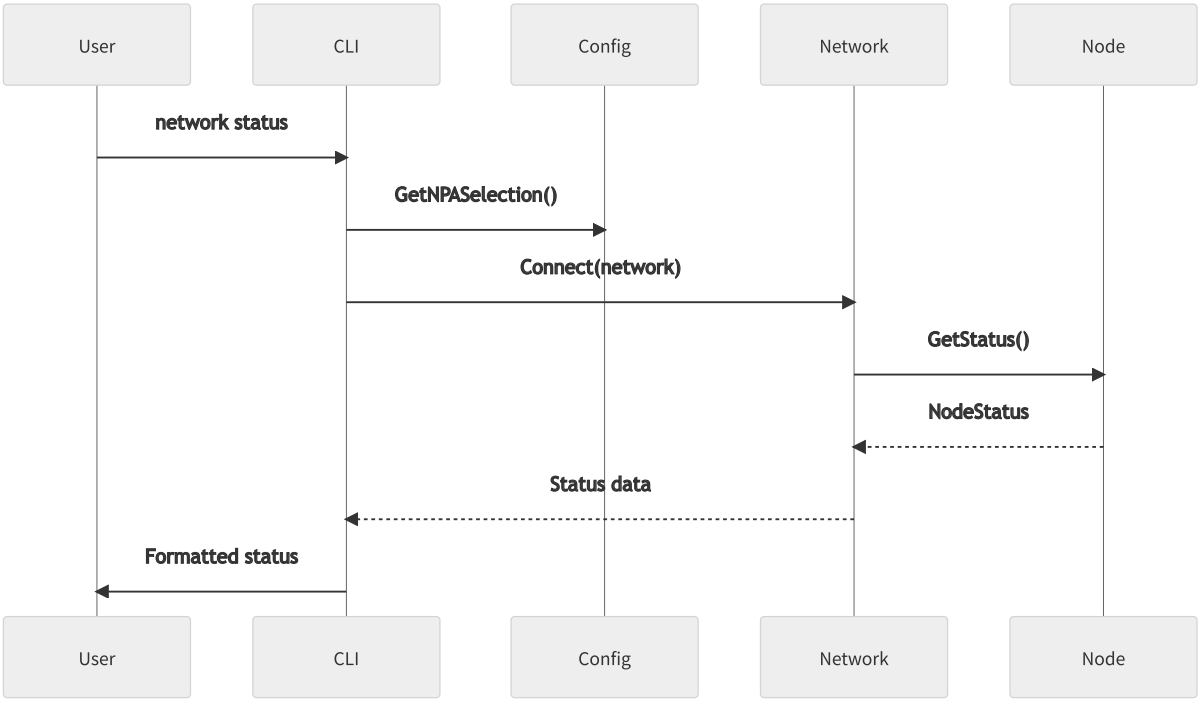
```
// 로컬 주소 검증

if !cmnGrpc.IsLocalAddress(net.RPC) {

cobra.CheckErr(fmt.Errorf("rpc-endpoint '%s' is not local", net.RPC))

}
```

데이터 흐름

네트워크 추가 흐름


User
CLI
Config
Network
Storage
network add <name> <rpc>
ValidateIdentifier(name)
ConnectNoVerify(rpc)
ChainContext
Survey for details
Network details
Networks.Add(name, net)
Save configuration
Success
Network added
User
CLI
Config
Network
Storage

상태 조회 흐름



성능 최적화

연결 관리

1. **연결 재사용**: 동일한 네트워크에 대한 연결 재사용
2. **타임아웃 설정**: 적절한 연결 타임아웃 설정
3. **에러 복구**: 일시적 네트워크 오류에 대한 재시도 로직

캐싱 전략

1. **체인 컨텍스트 캐싱**: 자주 조회되는 체인 컨텍스트 캐싱
2. **네트워크 정보 캐싱**: 네트워크 메타데이터 캐싱

3. **설정 캐싱**: 메모리 내 설정 캐싱

보안 고려사항

연결 보안

1. **TLS 검증**: 원격 연결에 대한 TLS 인증서 검증
2. **체인 컨텍스트 검증**: 올바른 네트워크 연결 확인
3. **로컬 연결 제한**: 로컬 네트워크 추가 시 경로 검증

설정 보안

1. **설정 파일 권한**: 적절한 파일 권한 설정
2. **입력 검증**: 모든 사용자 입력에 대한 검증
3. **경로 순회 방지**: 로컬 경로 처리 시 보안 검증

확장성 설계

새로운 네트워크 유형 지원

1. **프로토콜 확장**: 새로운 연결 프로토콜 지원
2. **설정 스키마**: 유연한 설정 스키마 설계
3. **마이그레이션 지원**: 새로운 네트워크 유형에 대한 마이그레이션

명령어 확장

1. **플러그인 아키텍처**: 새로운 네트워크 명령어 추가
2. **인터페이스 설계**: 일관된 명령어 인터페이스
3. **설정 통합**: 새로운 설정 옵션 통합

이 네트워크 관리 모듈은 Oasis CLI의 핵심 기능으로, 사용자가 다양한 Oasis 네트워크와 효율적으로 상호작용할 수 있도록 하는 견고하고 확장 가능한 시스템을 제공합니다.